PROGRAMAÇÃO WEB III

INTRODUÇÃO AO DJANGO

- Preparando Ambiente
- Configurar Time Zone
- Variáveis de Ambiente
- Views
- Urls
- Roteamento
- Templates
- Arquivos Estáticos
- Navegação
- DRY

PROF. ALEXANDRE ZORDAN DURÃES

PREPARANDO O AMBIENTE

Instalação do PYTHON

https://www.python.org/downloads/

Obs: Marcar opção: “Add python.exe to PATH”

PREPARANDO O AMBIENTE

Instalação do pacote PIP

Abra o prompt de comando ou terminal, de acordo com seu SO, e use os seguintes códigos

Windows

curl https://bootstrap.pypa.io/**get**-pip.py -o **get**-pip.py
python **get**-pip.py

MAC

curl https://bootstrap.pypa.io/**get**-pip.py -o **get**-pip.py
python3 **get**-pip.py

LINUX

sudo apt install python3-pip

PREPARANDO O AMBIENTE

Instalação do VirtualEnv

No prompt execute o seguinte código para instalar

pip install virtualenv

Virtualizando seu projeto

Acesse seu diretório pelo prompt e digite o seguinte código

python -m virtualenv venv

Ativando a venv

Windows: venv/Scripts/activate

MAC & LINUX: source venv/bin/activate

PREPARANDO O AMBIENTE

Instalando o Django

No prompt, dentro do seu ambiente virtualizado execute o seguinte código para instalar

pip install django

Registrando as dependências do projeto

pip freeze > requirements.txt

Criando projeto Django

django-admin startproject setup .

Startando o servidor

python manage.py runserver

PREPARANDO O AMBIENTE

PARABÉNS!

SEU AMBIENTE ESTÁ CONFIGURADO
E SEU SERVIDOR DJANGO ESTÁ RODANDO PELA PRIMEIRA VEZ!

CONFIGURANDO TIME ZONE

Vá até o arquivo settings.py

Altere o valor das seguintes propriedades

LANGUAGE_CODE = ‘pt-br’
TIME_ZONE para ‘America/Sao_Paulo’

Agora seu servidor está configurado.
Atualize o navegador e veja as mudanças!

PROTEGENDO VARÍAVEIS DE AMBIENTE

Instale o módulo

pip install python-dotenv

Atualize as dependências

pip freeze > requirements.txt

PROTEGENDO VARÍAVEIS DE AMBIENTE

Crie um arquivo na raiz do projeto chamado .env

Copie o valorda sua SECRET_KEY e cole dentro do seu arquivo .env

Ex: SECRET_KEY = django-insecure-(ioajsd¨%#@$_t-c&

Repare que o valor da KEY está sem ‘aspas’

Apague o valor da SECRET KEY do settings.py

PROTEGENDO VARÍAVEIS DE AMBIENTE

No cabeçalho do arquivo settings.py adicione as seguintes importações

```
import os
from pathlib import Path
from dotenv import load_dotenv
```

Vá até a variável SECRET_KEY e forneça o seguinte valor

```
SECRET_KEY = str(os.getenv('SECRET_KEY'))
```

PROTEGENDO VARÍAVEIS DE AMBIENTE

Crie um arquivo .gitignore na raiz do projeto

Acesse o site: gitignore.io
Procure por Django

Copie e cole o conteúdo em seu arquivo .gitignore

Pronto! Quando formos versionar no GitHub, por exemplo, estes arquivos não serão expostos!

CRIANDO UM APP

Um projeto Django pode ter diversos apps!

Para criar um app, use o comando:

python manage.py startapp nome_do_app

Para vincular um app ao projeto django devemos ir ao arquivo **settings.py (no projeto)** e na seção **INSTALLED APPS** adicionar o nome do nosso app **ao final da string**

Ex: 'nome_do_app',

CRIANDO UMA VIEW

No seu arquivo views.py (no app)

```
from django.http import HttpResponse

def index(request):
  return HttpResponse('<h1>Página Inicial</h1>')
```

Este código define que quando o servidor receber uma requisição no método index, será retornado uma responsa do tipo HttpResponse

ISOLANDO URLS

Crie o arquivo **urls.py** (do APP)

```
from django.urls import path
from nome_do_app.views import index

urlpatterns = [
  path('', index, name='index')
]
```

O atributo 'name' é uma espécie de identificador
o django vai se utilizar disso para navegar entre templates

ISOLANDO URLS

Edite o arquivo **urls.py** (do PROJETO)

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('nome_do_app.urls')),
]
```

TEMPLATES

No escopo do seu diretório raiz, no mesmo nível do APP e PROJETO, crie um diretório chamado **templates**

[Boa prática]
Isolar os templates por apps
neste caso dentro do diretório templates, criar outro com o nome_do_app

No arquivo settings.py
No escopo de TEMPLATES, na chave ‘DIRS’ altere para o seguinte valor
'DIRS': [os.path.join(BASE_DIR, 'templates')],

TEMPLATES

Crie um arquivo .html dentro do diretório templates
com o código de sua preferência

Altere seu arquivo views.py

```
from django.shortcuts import render

def index(request):
 return render(request, 'nome_do_app/index.html')
```

ARQUIVOS ESTÁTICOS

Para carregarmos arquivos estáticos ao Django devemos:

- Criar um diretório chamado **static** em nosso **PROJETO**
- Ir até o arquivo settings.py e **criar**, abaixo do STATIC_URL as seguintes diretivas:

```
STATICFILES_DIRS = [
  os.path.join(BASE_DIR, 'setup/static')
]

STATIC_ROOT = os.path.join(BASE_DIR, 'static')
```

ARQUIVOS ESTÁTICOS

Após colocarmos nossos arquivos estáticos (imagens, css, js, etc) dentro de nossos diretórios, precisamos sinalizar para o django que eles precisam ser **coletados**

No prompt

python manage.py collectstatic

Após o comando executado, o django vai mapear estes arquivos estáticos para um diretório estático na raiz do contexto

ARQUIVOS ESTÁTICOS

Para que o nosso documento HTML entenda que nós queremos utilizar os arquivos estáticos precisamos das seguintes diretivas:

Na primeira linha do HTML

{% load static %}

Editar referência com o css e indicar que agora é estático

<link rel="stylesheet" href="{% static '/styles/style.css' %}">

Para as imagens, devemos fazer a mesma edição na referência

Ex: <img src="{% static '/caminho-da-imagem' %}"

OUTRAS PÁGINAS

Para que o django possa manipular outras páginas HTML, devemos incluí-las no diretório de templates, no subdiretório da aplicação

Definir em **views.py** uma função para renderizar esta página

```
def outraPagina(request):
    return render(request, 'nome_do_app/outraPagina.html')
```

OUTRAS PÁGINAS

Depois da função definida, devemos mapear sua rota em **urls.py**

Realizar a importação da view configurada

from nome_do_app.views importe index, **outraPagina**

Mapear em nossas rotas a nova rota definida

```
urlpatterns = [
    path('', index, name='index'),
    path('nome_da_rota','outraPagina, name='name-da-pagina')
]
```

Utilizar código django em nosso link, apontando para o 'name'

```
<a href="{% url 'name' %}"></a>
```

DRY

DONT REPEAT YOURSELF

Este padrão de boas práticas
visa nos atentar sobre reutilização de códigos
sempre que possível, evitando redundâncias e
facilitando a manutenção do código.

Podemos aplicar DRY em também nossos templates!
Imagine que nossas páginas tenham diversas estruturas idênticas ...

Podemos criar um arquivo contendo este bloco de HTML e apenas realizar sua chamada.

APLICANDO DRY

No diretório de templates do seu **app,**
crie um arquivo chamado **base.html**

Cole o código base que será utilizado (repetido)

Na primeira linha do faremos o uso do carregamento estático do django
{% load static %}

Dentro do <body> </body> informamos ao django que este conteúdo será preenchido por um bloco externo
{% block content %}{% endblock %}

APLICANDO DRY

base.html

templates > galeria > base.html > html > head

```html
{% load static %}
<!DOCTYPE html>
<html lang="pt-br">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Django Space</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="{% static '/styles/style.css' %}">

</head>

<body>
    {% block content %}{% endblock %}
</body>
</html>
```

APLICANDO DRY

Nos templates que utilizarão do carregamento desta base
retiramos todo o código que será substituído e
incluímos as seguintes diretivas no início do arquivo

```
{% extends 'nome_do_app/base.html' %}
{% load static %}
{% block content %}
```

É preciso informar ao django o final do bloco
para isso, no final do arquivo utilizamos a seguinte diretiva

```
{%endblock%}
```

APLICANDO DRY

Além dos conteúdos estruturais, podemos aplicar o DRY para "componentizar" conteteúdos parciais, como rodapés e navbars que são muitas vezes repetidos em várias páginas

Para os conteúdos parciais, é **boa prática** criar um diretório chamado **partials** para a organização destes.

Nestes casos, precisamos informar ao django que este conteúdo parcial terá o carregamento estático, na primeira linha do arquivo
{% load static %}

APLICANDO DRY

No caso de um **footer**, que se repete em todos os templates, podemos criar um arquivo **_footer.html** em nosso diretório partials e utilizá-lo diretamente na nossa estrutura **base.html**

Para isso, basta incluir no final do body a diretiva
{% include 'nome_do_app/partials/_footer.html' %}

APLICANDO DRY

No caso de um **componente** que não se repete sempre,
mas pode aparecer mais de uma vez, como por exemplo um menu lateral,
podemos criar seu arquivo em nosso diretório partials **_menu.html**
incluindo no início do arquivo html a seguinte diretiva:

```
{% load static %}
```

depois, no arquivo html de destino, basta inserir a seguinte diretiva
no local desejado para realizar a inclusão do componente

```
{% include 'nome_do_app/partials/_menu.html' %}
```